

Sistema de Filtros — Listado de Músicos

Índice

- 1. Descripción general
- 2. Arquitectura y tecnologías implicadas
- 3. Flujo completo de datos
- 4. Capa de presentación — Twig
- 5. Autocompletado de ciudad — Stimulus + Google Places
- 6. Geolocalización del navegador
- 7. Slider de radio de proximidad
- 8. Distancia en la tarjeta del músico
- 9. Capa de control — MusicoController
- 10. Capa de acceso a datos — MusicoRepository
- 11. Fórmula de Haversine
- 12. Estructura de parámetros GET
- 13. Casos de uso y comportamiento esperado
- 14. Diagrama de flujo

1. Descripción general

El sistema de filtros permite al usuario encontrar músicos según tres criterios combinables:

Filtro	Tipo	Descripción
Género musical	Multisección (checkboxes)	Filtra músicos que toquen al menos uno de los géneros seleccionados
Instrumento	Multisección (checkboxes)	Filtra músicos que toquen al menos uno de los instrumentos seleccionados
Proximidad geográfica	Campo de texto + slider	Filtra músicos cuyo perfil esté dentro de un radio (km) respecto a una ubicación introducida

Los tres filtros son **acumulativos**: si se activan varios, el resultado devuelve únicamente los músicos que cumplan todos a la vez.

La búsqueda es **sin estado en servidor**: todos los parámetros viajan en la URL mediante una petición **GET**, lo que permite compartir o guardar en favoritos cualquier búsqueda.

2. Arquitectura y tecnologías implicadas

```
Navegador (Frontend)
|
|— Twig ..... templates/musico/index.html.twig
|      Renderiza el formulario de filtros y el grid de tarjetas
```

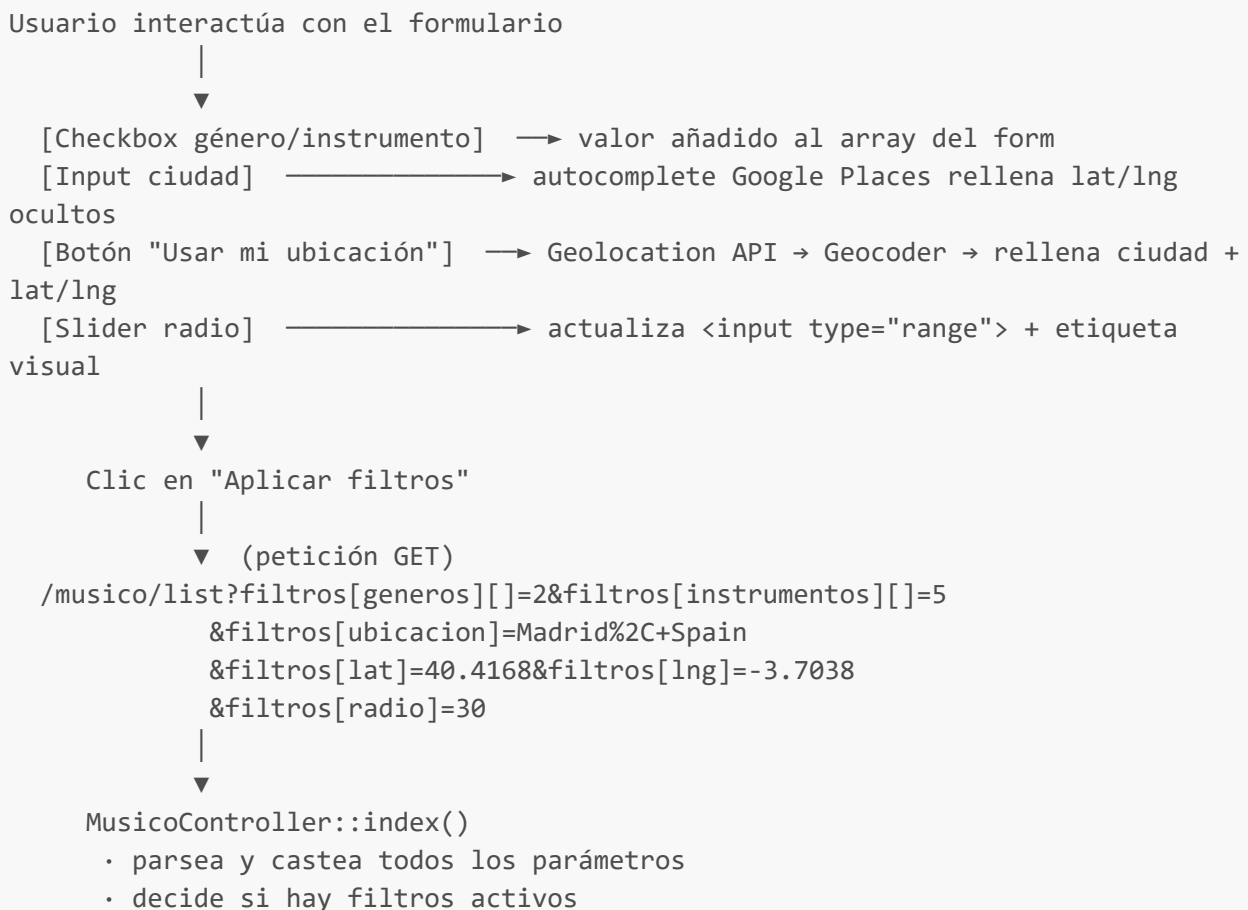
- CSS public/css/perfil.css
Estilos del sidebar, checkboxes, slider y botón de geolocalización
- JavaScript (inline)
 - Slider de radio: actualiza el degradado y la etiqueta en tiempo real
 - Geolocalización: Geolocation API → reverse geocoding → rellena el formulario
- Stimulus (assets/controllers/google-places-autocomplete_controller.js)
Carga la API de Google Places y conecta el autocomplete al campo de texto

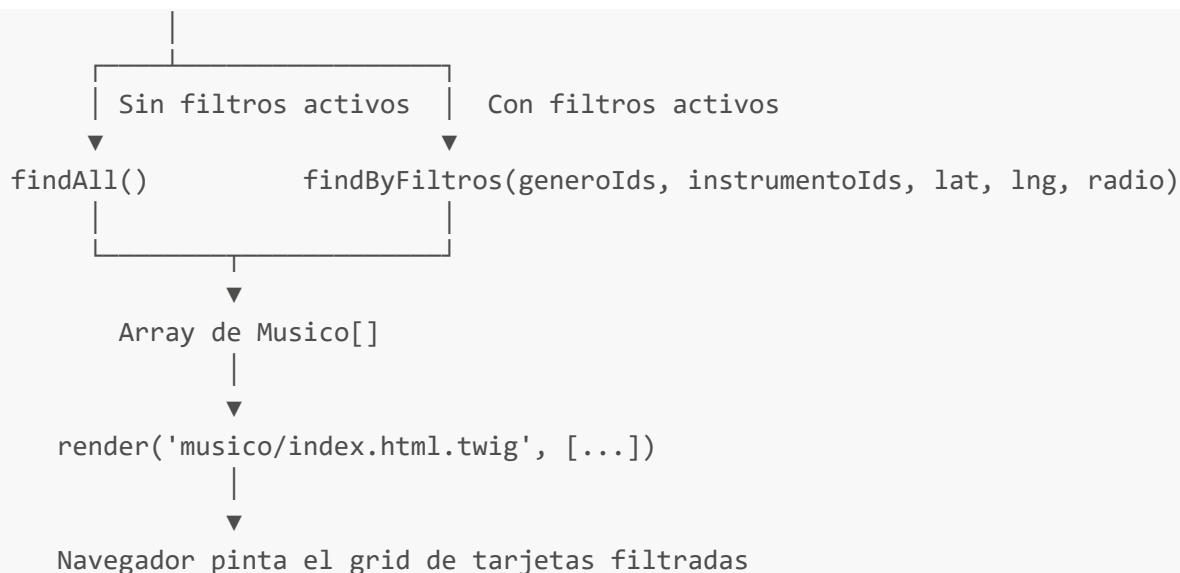
Servidor (Backend)

- MusicoController · src/Controller/MusicoController.php
Recibe los parámetros GET, los parsea y decide qué query ejecutar
- MusicoRepository · src/Repository/MusicoRepository.php
Consulta a la base de datos (géneros e instrumentos) y aplica el filtro de distancia en PHP mediante la fórmula de Haversine

Stack: Symfony 7, Doctrine ORM, Twig, Stimulus (Symfony UX), Google Maps JavaScript API (Places + Geocoder), API de Geolocalización del navegador (W3C).

3. Flujo completo de datos





4. Capa de presentación — Twig

Archivo: `templates/musico/index.html.twig`

El formulario usa `method="get"` y como `action` la ruta `app_musico_index`. Al no ser `POST`, los datos viajan directamente en la URL y no se necesita ningún token CSRF.

4.1 Checkboxes (géneros e instrumentos)

```

<input type="checkbox"
  name="filtros[generos][]"
  value="{ { genero.id } }"
  { % if genero.id in (filtros.generos ?? []) % }checked{ % endif % }>

```

- El nombre `filtros[generos][]` hace que PHP interprete los valores como un array:
`$_GET['filtros']['generos'] = [1, 3, 7]`.
- La condición `{ % if genero.id in (filtros.generos ?? []) % }` restaura el estado marcado cuando la página se recarga tras aplicar filtros, usando el array `$filtros` que el controlador reenvía a la vista.

El controlador pasa a la plantilla **todos** los géneros e instrumentos del sistema para renderizar la lista, y también pasa el array `filtros` con los valores seleccionados actualmente.

4.2 Campo de ubicación

```

<div data-controller="google-places-autocomplete"
  data-google-places-autocomplete-api-key-value="{ { google_places_api_key } }">

  <input class="filtros-ubicacion-input" type="text" ...> { # visible, solo UI
#}

```

```

    <input type="hidden" name="filtros[ubicacion]" ...>           {# nombre legible
  #}
    <input type="hidden" name="filtros[lat]"
      data-google-places-autocomplete-target="lat">           {# latitud decimal
  #}
    <input type="hidden" name="filtros[lng]"
      data-google-places-autocomplete-target="lng">           {# longitud decimal
  #}
</div>

```

El campo de texto visible **no** tiene **name**, por lo que su valor no se envía al servidor. Los datos que realmente viajan son los tres **hidden**: texto de ubicación (para mostrar en el form al recargar), latitud y longitud.

4.3 Slider de radio

```

<input type="range" name="filtros[radio]"
  min="10" max="100" step="10"
  value="{ { filtros.radio ?? '20' } }">

```

- Rango: 10–100 km en pasos de 10.
- Valor por defecto: **20 km** (si no hay filtro activo en la URL).
- El atributo **value** toma el valor del array `$filtros` para restaurar la posición del slider al recargar.

5. Autocompletado de ciudad — Stimulus + Google Places

Archivo: `assets/controllers/google-places-autocomplete_controller.js`

¿Qué es Stimulus?

Stimulus es un framework JavaScript minimalista integrado en Symfony UX. Conecta controladores JS a elementos HTML mediante atributos `data-controller`, `data-*-target` y `data-action`, sin necesidad de manipular el DOM directamente.

Ciclo de vida del controlador

```

connect() → ¿Google Maps ya cargado?
           | Sí → initAutocomplete()
           | No → loadScript() → [script cargado] → initAutocomplete()

```

loadScript()

Inyecta dinámicamente en el `<head>` la siguiente URL:

```

https://maps.googleapis.com/maps/api/js
?key=<API_KEY>

```

```
&libraries=places
&language=es
```

Se comprueba si ya existe un script con `id="google-maps-script"` para no cargarlo dos veces si hay varios controladores en la página.

`initAutocomplete()`

Crea una instancia de `google.maps.places.Autocomplete` sobre el `<input type="text">` visible, configurada con:

- `types: ['(regions)']` — solo sugiere regiones/ciudades, no direcciones exactas.
- `fields: ['address_components', 'formatted_address', 'geometry']` — limita los campos devueltos para reducir el coste de la API.

Cuando el usuario elige una sugerencia (`place_changed`):

1. Extrae el nombre de la ciudad y el país de `address_components`.
2. Escribe "`Ciudad, País`" en el campo de texto visible.
3. Escribe la latitud en el target `lat` y la longitud en el target `lng`.

El resultado son coordenadas decimales precisas (p.ej. `40.4168`, `-3.7038`) que el backend usa para el cálculo de distancia.

6. Geolocalización del navegador

Código: bloque `<script>` al final de `templates/musico/index.html.twig`

El botón "**Usar mi ubicación**" usa la [Geolocation API](#) del W3C, disponible en todos los navegadores modernos. Solo funciona en contextos seguros (HTTPS o localhost).

Flujo

```

Clic en el botón
|
▼
navigator.geolocation.getCurrentPosition(éxito, error)
|
▼ (éxito – el usuario concede permiso)
pos.coords.latitude / pos.coords.longitude
|
├─> Rellena filtros[lat] y filtros[lng] (inputs hidden)
|
└─> ¿window.google disponible?
    |
    | Sí → new google.maps.Geocoder().geocode({ location })
    |       → extrae ciudad + país → rellena campo de texto
    |       y filtros[ubicacion]
    |
    | No → escribe "lat, lng" como texto de ubicación
    |       (el filtro de distancia funciona igual)
  
```

▼ (error – permiso denegado o fallo)
 Botón vuelve a su estado original, sin mensaje de error visible

Por qué se necesita el reverse geocoding

El backend no usa el texto de `filtros[ubicacion]` para calcular distancias: solo usa `lat` y `lng`. El texto de ciudad se escribe únicamente para que el campo de texto muestre algo legible al usuario cuando la página se recarga.

7. Slider de radio de proximidad

Código: bloque `<script>` al final de `templates/musico/index.html.twig`

```
function updateSlider(v) {
    var pct = (v - 10) / 90 * 100;    // convierte valor a porcentaje de la barra
    slider.style.background =
        'linear-gradient(to right, #9b30ff ' + pct + '%, rgba(155,48,255,0.18) ' +
    pct + '%)';
    label.textContent = v + ' km';
}
```

- El rango va de 10 a 100, por lo que el **rango útil es 90 unidades**.
- La fórmula $(v - 10) / 90 * 100$ normaliza el valor a un porcentaje entre 0% (10 km) y 100% (100 km).
- Se aplica como fondo del propio `<input>` con `linear-gradient`, dividiendo la barra en zona activa (púrpura) y zona inactiva (púrpura tenue).

7.2 Slider deshabilitado sin ubicación

El slider de radio no tiene sentido sin una ubicación de referencia: sin `lat/lng` el backend lo ignora. Para evitar que el usuario interactúe con un control que no tiene efecto, el slider se deshabilita visualmente cuando el campo de ciudad está vacío.

El problema de los eventos sintéticos

El campo de texto de ubicación se actualiza de **dos formas programáticas** que no disparan el evento `input` nativo del DOM:

Origen	Cómo actualiza el texto	Solución aplicada
Google Places autocomplete (Stimulus)	<code>input.value = 'Madrid, Spain'</code>	<code>input.dispatchEvent(new Event('input'))</code> añadido en <code>place_changed</code>
Botón "Usar mi ubicación"	<code>textInput.value = label</code> dentro de <code>setLabel()</code>	<code>textInput.dispatchEvent(new Event('input'))</code> añadido en <code>setLabel()</code>

Origen	Cómo actualiza el texto	Solución aplicada
El usuario escribe manualmente	Dispara <code>input</code> de forma nativa	No requiere cambio
El usuario borra el texto	Dispara <code>input</code> de forma nativa	No requiere cambio

Con ese despacho manual, **todos los caminos de escritura** convergen en el mismo listener `input` del campo de texto.

Lógica de sincronización

```
function syncSlider() {
  var hasLocation = ubicacionInput.value.trim() !== '';
  slider.disabled = !hasLocation;
  slider.closest('.filtros-radio-wrap')
    .classList.toggle('filtros-radio-disabled', !hasLocation);
  if (!hasLocation) {
    slider.style.background = 'rgba(155,48,255,0.18)'; // barra completamente
    apagada
  } else {
    updateSlider(+slider.value); // restaura el
    degradado
  }
}
ubicacionInput.addEventListener('input', syncSlider);
syncSlider(); // estado inicial al cargar la página
```

- `syncSlider()` se llama también en la carga inicial para que, si hay una ubicación ya en la URL, el slider aparezca habilitado desde el principio.
- `slider.disabled = true` hace que el valor del input **no se envíe** en el formulario GET, lo que es correcto: sin ubicación no debe mandarse el radio.
- La clase `.filtros-radio-disabled` en el wrapper aplica `opacity: 0.35` y `pointer-events: none` para dar retroalimentación visual clara.

CSS

```
.filtros-radio-wrap.filtros-radio-disabled {
  opacity: 0.35;
  pointer-events: none;
  cursor: not-allowed;
}
```

8. Distancia en la tarjeta del músico

Cuando el filtro de proximidad está activo, cada tarjeta muestra la distancia exacta desde el punto de búsqueda hasta el músico, junto al resto de metadatos (ciudad, años de experiencia).

Diseño de la solución

El cálculo ya existe en el repositorio (método `haversine`). La decisión de diseño fue **no alterar la firma de `findByFiltros()`** para no romper nada, y calcular las distancias en el controlador como un array paralelo, independiente del array de músicos.

```
findByFiltros() → Musico[]           (sin cambios)
calcularDistanciaKm() × N → int[] $distancias  (nuevo, paralelo)
```

Flujo

```
MusicoController::index()
├── findByFiltros(...) → $musicos[]
├── ¿$lat !== null && $lng !== null?
│   └── Sí
│       └── foreach $musicos as $m
│           ├── ¿$m tiene lat/lng guardados?
│           │   ├── Sí → $distancias[$m->getId()] = round(haversine(...), 1)
│           │   └── No → no se añade entrada (tarjeta no mostrará distancia)
│           └── render(..., ['distancias' => $distancias])
```

Código del controlador

```
$distancias = [];
if ($lat !== null && $lng !== null) {
    foreach ($musicos as $m) {
        if ($m->getLatitud() !== null && $m->getLongitud() !== null) {
            $distancias[$m->getId()] = round(
                $musicoRepository->calcularDistanciaKm($lat, $lng, $m-
>getLatitud(), $m->getLongitud()),
                1
            );
        }
    }
}
```

- `$distancias` es un array asociativo [`id => km`], por ejemplo [`3 => 12.4`, `7 => 28.0`].
- La clave es el ID del músico, lo que permite a la plantilla hacer lookup en O(1).
- Cuando no hay filtro de ubicación el array llega vacío a la plantilla y ninguna tarjeta muestra distancia.

Método público en el repositorio

Para hacer accesible el cálculo desde el controlador sin duplicar la fórmula, se expuso un método público que delega en el privado `haversine`:

```
public function calcularDistanciaKm(float $lat1, float $lng1, float $lat2, float $lng2): float
{
    return $this->haversine($lat1, $lng1, $lat2, $lng2);
}
```

Plantilla Twig

```
{% if musico.id in distancias|keys %}
    <span class="musico-card-distancia-tag">{{ distancias[musico.id] }} km</span>
{% endif %}
```

- `distancias|keys` devuelve el array de IDs existentes; la comprobación `in` evita errores si el músico no tiene coordenadas.
- El span se inserta dentro de `.musico-card-tags`, por lo que hereda automáticamente el separador • que ya existe entre los otros metadatos.
- Si `$distancias` está vacío (sin filtro de ubicación), el bloque nunca se renderiza y la tarjeta no cambia.

CSS

```
.musico-card-distancia-tag {
    color: var(--mh-cyan) !important;
    font-weight: 600;
}
```

Color cian más intenso y negrita para diferenciar visualmente la distancia calculada del resto de metadatos (ciudad, años de experiencia), que usan `var(--mh-cyan-soft)`.

9. Capa de control — MusicoController

Archivo: `src/Controller/MusicoController.php`

Ruta: `GET /musico/list` → `app_musico_index`

Parseo de parámetros

```
$filtros = $request->query->all('filtros');
```

`query->all('filtros')` devuelve el subárbol `filtros[*]` del query string como array PHP asociativo, o un array vacío si no existe.

A continuación se castean y validan los valores:

```
$generoIds      = !empty($filtros['generos'])      ? array_map('intval', ...) :
[];
$instrumentoIds = !empty($filtros['instrumentos']) ? array_map('intval', ...) :
[];
$lat    = isset($filtros['lat'])  && $filtros['lat']  !== '' ? (float)
$filtros['lat'] : null;
$lng    = isset($filtros['lng'])  && $filtros['lng']  !== '' ? (float)
$filtros['lng'] : null;
$radio  = isset($filtros['radio']) && $filtros['radio'] !== '' ? (int)
$filtros['radio'] : null;
```

- Los IDs se castean a `int` para evitar inyección SQL aunque Doctrine los parametriza.
- `lat` y `lng` se castean a `float` (coordenadas decimales).
- `radio` se castea a `int` (kilómetros).
- Cualquier campo ausente o vacío resulta en `null` / array vacío, nunca en un error.

Decisión de consulta

```
$hayFiltros = !empty($generoIds) || !empty($instrumentoIds) || ($lat !== null &&
$radio !== null);

$musicos = $hayFiltros
    ? $musicoRepository->findByFiltros(...)
    : $musicoRepository->findAll();
```

Si no hay ningún filtro activo se evita la consulta compleja y se usa `findAll()` directamente.

Nota sobre el filtro de ubicación: para que el filtro geográfico se active es necesario tener **tanto** `lat`/`lng` **como** `radio`. Tener solo la ciudad en texto no activa nada; el texto de `filtros[ubicacion]` solo se reenvía a la vista para repopular el campo.

Datos enviados a la plantilla

```
return $this->render('musico/index.html.twig', [
    'musicos'      => $musicos,           // resultado filtrado
    'generos'      => $generoRepository->findBy([], ['nombre' => 'ASC']),
    'instrumentos' => $instrumentoSistemaRepository->findBy([], ['nombre' =>
'ASC']),
    'filtros'      => $filtros,           // array original para repopular el form
]);
```

El array `$filtros` se devuelve sin modificar a la plantilla para que los checkboxes y el slider muestren el estado actual al recargar la página.

10. Capa de acceso a datos — MusicoRepository

Archivo: `src/Repository/MusicoRepository.php`

`findByFiltros()`

```
public function findByFiltros(
    array $generoIds,
    array $instrumentoIds,
    ?float $lat,
    ?float $lng,
    ?int $radio
): array
```

Fase 1 — Filtro en base de datos (Doctrine QueryBuilder)

```
$qb = $this->createQueryBuilder('m');

if (!empty($generoIds)) {
    $qb->join('m.generosMusicales', 'g')
        ->andWhere('g.id IN (:generos)')
        ->setParameter('generos', $generoIds);
}

if (!empty($instrumentoIds)) {
    $qb->join('m.instrumentosSistema', 'i')
        ->andWhere('i.id IN (:instrumentos)')
        ->setParameter('instrumentos', $instrumentoIds);
}

$musicos = $qb->distinct()->getQuery()->getResult();
```

- Cada condición añade un `JOIN` y un `AND WHERE ... IN (...)`.
- `distinct()` evita duplicados cuando un músico cumple varios géneros/instrumentos a la vez.
- Si `$generoIds` y `$instrumentoIds` están vacíos, la query recupera todos los músicos (el filtro geográfico se aplica en la fase 2).

Fase 2 — Filtro geográfico en PHP

```
if ($lat !== null && $lng !== null && $radio !== null && $radio > 0) {
    $musicos = array_values(array_filter($musicos, function (Musico $m) use ($lat,
    $lng, $radio) {
```

```

        if ($m->getLatitud() === null || $m->getLongitud() === null) {
            return false;
        }
        return $this->haversine($lat, $lng, $m->getLatitud(), $m->getLongitud())
    <= $radio;
    }));
}

```

- Los músicos sin coordenadas guardadas en su perfil son **excluidos** del resultado cuando hay filtro geográfico.
- `array_values()` reindexación el array tras el filtrado para evitar índices discontinuos.

Por qué en PHP y no en SQL: el cálculo de Haversine con SQL puro requiere funciones trigonométricas y difiere entre motores (MySQL, PostgreSQL, SQLite). Hacerlo en PHP mantiene la portabilidad y simplifica el código, asumiendo que el número de músicos cargados en memoria es razonable para una aplicación del tamaño de este TFG.

11. Fórmula de Haversine

La fórmula de Haversine calcula la **distancia ortodrómica** (la más corta sobre la superficie esférica de la Tierra) entre dos puntos dados por sus coordenadas geográficas (latitud y longitud en grados).

Implementación

```

private function haversine(float $lat1, float $lng1, float $lat2, float $lng2):
float
{
    $dLat = deg2rad($lat2 - $lat1);
    $dLng = deg2rad($lng2 - $lng1);
    $a = sin($dLat / 2) ** 2
        + cos(deg2rad($lat1)) * cos(deg2rad($lat2)) * sin($dLng / 2) ** 2;
    return 6371 * 2 * atan2(sqrt($a), sqrt(1 - $a));
}

```

Explicación paso a paso

Variable	Significado
<code>\$dLat</code>	Diferencia de latitud en radianes
<code>\$dLng</code>	Diferencia de longitud en radianes
<code>\$a</code>	Parámetro intermedio (cuadrado del semiseno del ángulo central)
<code>6371</code>	Radio medio de la Tierra en kilómetros
<code>atan2(√a, √(1-a))</code>	Ángulo central en radianes
Resultado	Distancia en kilómetros

Fórmula matemática

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cdot \cos\phi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$
$$d = 2R \cdot \arctan 2(\sqrt{a}, \sqrt{1-a})$$

donde ϕ es la latitud, λ la longitud y $R = 6\,371$ km.

Precisión

El error es inferior al 0,3 % para la mayoría de distancias terrestres. Para el propósito de este sistema (buscar músicos en un radio de decenas de km) la precisión es más que suficiente.

12. Estructura de parámetros GET

Una URL de ejemplo con todos los filtros activos:

```
/musico/list
?filtros[generos][]=2
&filtros[generos][]=5
&filtros[instrumentos][]=3
&filtros[ubicacion]=Madrid%2C+Spain
&filtros[lat]=40.4168
&filtros[lng]=-3.7038
&filtros[radio]=30
```

Parámetro	Tipo PHP	Uso
<code>filtros[generos][]</code>	<code>int[]</code>	IDs de géneros a filtrar (puede repetirse)
<code>filtros[instrumentos][]</code>	<code>int[]</code>	IDs de instrumentos a filtrar (puede repetirse)
<code>filtros[ubicacion]</code>	<code>string</code>	Texto legible de la ciudad (solo visual, no se usa en la query)
<code>filtros[lat]</code>	<code>float</code>	Latitud del punto de referencia
<code>filtros[lng]</code>	<code>float</code>	Longitud del punto de referencia
<code>filtros[radio]</code>	<code>int</code>	Radio en km (10–100, pasos de 10)

13. Casos de uso y comportamiento esperado

Solo géneros seleccionados

Se genera una query con `JOIN generosMusicales` y `WHERE g.id IN (...)`. No se aplica filtro geográfico. Se devuelven todos los músicos con al menos uno de esos géneros.

Solo instrumento seleccionado

Análogo al caso anterior pero con `JOIN instrumentosSistema`.

Género + instrumento

Ambos `JOIN` se encadenan con `AND WHERE`, devolviendo únicamente los músicos que cumplen los dos criterios a la vez.

Ubicación sin radio (o radio sin ubicación)

No se activa el filtro geográfico. La comprobación `$lat !== null && $radio !== null` exige que ambos estén presentes. La ubicación en texto sola no tiene efecto.

Ubicación + radio (sin género ni instrumento)

La query base recupera todos los músicos (`findAll` equivalente) y luego se filtra en PHP por distancia.

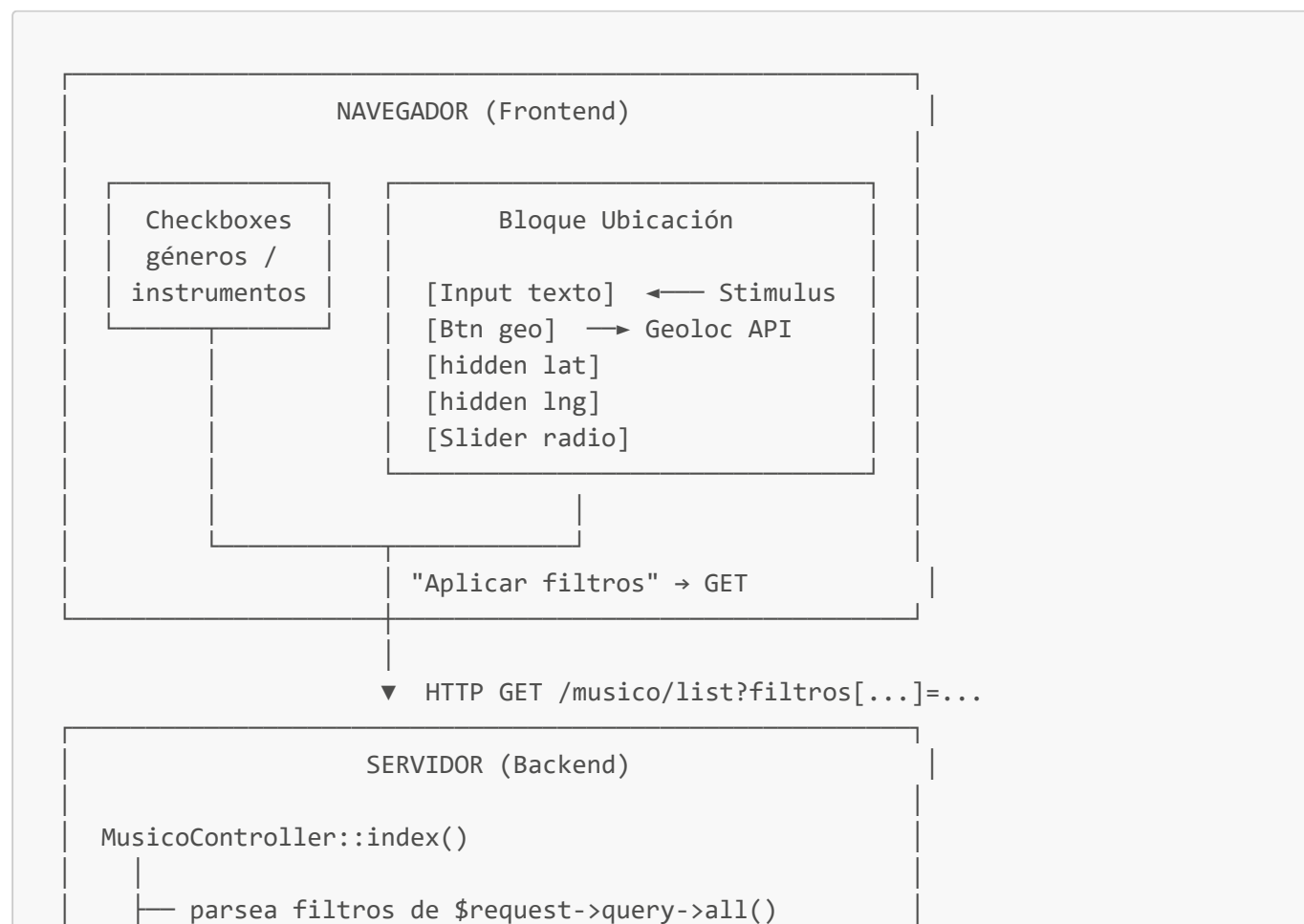
Todos los filtros activos

Query con ambos `JOIN` → resultado → filtrado adicional por Haversine. El resultado son músicos que tocan esos géneros, esos instrumentos Y están dentro del radio indicado.

Sin ningún filtro

Se llama directamente a `findAll()`, evitando la query más costosa.

14. Diagrama de flujo



```
|— castea tipos (intval, float, int)
|— $hayFiltros ?
    | No → findAll()
    | Sí → findByFiltros(...)
```

MusicoRepository

```
QueryBuilder
JOIN géneros
JOIN instrum.
DISTINCT
```

Musico[]

{lat + radio activos?

```
| Sí → array_filter Haversine
| No → resultado tal cual
```

render('musico/index.html.twig')

▼ HTML con grid de tarjetas filtradas

NAVEGADOR (respuesta)

- Grid muestra solo los músicos que pasan todos los filtros
- Checkboxes y slider restaurados con los valores de la URL (repopulación desde \$filtros)